QUALITY ASSURANCE TESTING DOCUMENTATION

(QATD)

UMHackathon 2026

Domain 2

Prepared by: Fantastic 5

Team Members:

1. Bibianne Zheyee Joseph
2. Hang Xiu Jun
3. Kishanea A/P Jeyakumar
4. Liew Jia Xin
5. Ng Guan Yik

Table of Content

Document Control 3

PRELIMINARY ROUND (Test Strategy & Planning) 4

1. Scope & Requirements Traceability 4

   1.1 In-Scope Core Features 4

   1.2 Out-of-Scope 4

2. Risk Assessment & Mitigation Strategy 5

3. Test Environment & Execution Strategy 7

4. CI/CD Release Thresholds & Automation Gates 9

   4.1 Integration Thresholds (Merging to Main) 9

   4.2 Deployment Thresholds (Pushing to Production) 9

5. Test Case Specifications (Drafts) 10

6. AI Output & Boundary Testing (Drafts) 11

   6.1 Prompt/Response Test Pairs 11

   6.2 Oversized/Larger Input Test 11

   6.3 Adversarial/Edge Prompt Test 11

   6.4 Hallucinating Handling 12

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 – Team Fantastic 5
Team Repo URL	https://github.com/meowkemp/UMHackathon.git
Project Board URL	
Live Deployment URL (optional for preliminary round)	

Objective:

The primary objective is to ensure that FInSight AI can accurately evaluate user financial inputs, generate reliable AI-based recommendations, simulate financial scenarios and provide explainable outputs while maintaining performance, reliability and correctness under different conditions.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns the testing with user requirements to ensure all core functionalities are validated and no feature behaves unexpectedly.

1.1 In-Scope Core Features

- Purchase Decision Advisor:
  Evaluate purchase and return Buy / Delay / Reconsider
- Scenario Simulation Engine:
  Simulate 12-month financial outcomes
- AI Explanation Engine:
  Generate explainable recommendations using GLM
- Receipt Scanner (OCR):
  Extract purchase data and trigger analysis
- Peer Benchmarking:
  Compare user behavior with dataset

1.2 Out-of-Scope

- Banking API integration
- Multi-user collaboration
- Mobile application
- Real-time financial syncing

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
AI generates incorrect recommendation	3	5	15 (High)	Use rule-based validation and structured prompts.	Compare AI output with expected logic.
OCR extracts wrong price	3	4	12(High)	Add manual correction input.	Test with multiple receipt images.
Simulation produces inaccurate projection	2	4	8 (Medium)	Validate formula expected outcomes.	Cross-check with manual calculations.
API latency too slow (>5s)	4	3	12 (High)	Optimize prompt size and async calls.	Load testing with multiple requests.
Missing user data input	4	2	8 (Medium)	Input validation checks.	Negative testing.

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact

4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	Risk Level	Recommended Action
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and testing is required
16 – 25	Critical	Priority is Highest. Need extensive level of testing.

3. Test Environment & Execution Strategy

- **Unit Test**
  - Scope:
    Decisions logic (BUY / DELAY / RECONSIDER) which is the simulation calculations.
  - Execution:
    PyTest during development and CI pipeline.
  - Isolation:
    External dependencies (API, OCR) are mocked.
  - Pass-Condition:
    Correct decision output and valid edge case handling.

- **Integration Test**
  - Scope:
    UI → backend → AI → output flow
  - Execution:
    After feature integration
  - Workflow:
    Real API calls tested
  - Pass Condition:
    End-to-end flow produces correct results

- **Test Environment (CI/CD Practice):**
  - Local Testing:
    Developer machine
  - Staging:
    GitHub Actions pipeline
  - CI/CD:
    Automated testing to push and pull requests

- **Regression Testing & Pass/Fail Rules:**

- Execution Phase:
  After each feature update.
- Pass Condition:
  Output matches expected result
- Continuation Rule:
  Critical failures must be fixed before proceeding

- **Test Data Strategy:**
  - Manual:
    Sample financial profiles such as low income, high income, high spender
  - Automated:
    Predefined datasets for benchmarking

- **Passing Rate Threshold:**
  - Minimum 85% test pass rate
  - Critical tests must achieve 100% pass

4. CI/CD Release Thresholds & Automation Gates

4.1 Integration Thresholds (Merging to Main)

Checks	Requirements	Project	Pass/Failed
Automatic Build	Zero Build Error	0 Errors	Passed
Unit Tests	100% Passing Rate	100%	Passed
Code Quality	Zero Linting Error	98%	Passed
Test Coverage	Minimum 80 %	85%	Passed

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	Minimum 90%	92%	Passed
AI Output Pass Rate	≥ 80% valid outputs	85%	Passed
Critical Bugs	Zero P0/01 Bugs	0	Passed
API Performance	Response Time < 800ms	650ms	Passed
Security	API keys are not exposed	Clean	Passed

5. Test Case Specifications (Drafts)

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	**Happy Case (Entire Flow):** Purchase Decision	Verify full flow from input → AI decision → explanation	1. Enter income, savings, and item price. 2. Submit for evaluation. 3. View recommendation and explanation	System returns correct decisions and explanation.	Decision generated correctly with explanation. **Status: Passed**
TC-02	**Specific Case (Negative):** Missing Input	Check the system behaviors when required fields are missing.	1. Leave income field empty. 2. Submit the form.	System shows validation error.	Error displayed but lacks detailed guidance. **Status: Failed**
TC-03	**NFR (Performance):** API Response Time	Test system response under load.	1. Send 50 concurrent requests. 2. Measure the response time.	Average response time < 800ms.	Average Response time is received as 920ms. **Status: Failed**
TC-04	**NFR (Load):** Simulation Stability	Test Simulation under repeated usage.	1. Run simulation 100 times. 2. Monitor consistency.	Consistent outputs.	Stable outputs. **Status: Passed**

6. AI Output & Boundary Testing (Drafts)

6.1 Prompt/Response Test Pairs

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	"Should I buy a RM3000 phone with RM 2000 savings?"	Clean recommendation with reasoning	Valid explanatio n	Passed
AI-02	JSON financial data input	Accurate structured output.	Correct mapping	Passed
AI-03	"I want to buy but no sure."	Context-aware advice	Slightly vague	Failed

6.2 Oversized/Larger Input Test

Fields	Details
Maximum Input Size	1500 tokens
Input used while testing	6000 words
Expected Behavior	Chunking
Actual Behavior	Input truncated
Status	Passed

6.3 Adversarial/Edge Prompt Test

- Prompt:

  "I have no money but want to buy everything."
- Result:

System suggests not proceeding and provides financial advice.

- Handling

  Safe fallback recommendation.

6.4 Hallucinating Handling

- Structured prompts reduce hallucination.
- Rule-based validation checks outputs.
- Low-confidence response flagged.